C++ OOP LAB ASSIGNMENT

Top 25 Programs with Code and Output

Q1. Hello World

Objective: First basic program

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

OUTPUT:

```
Hello, World!
```

Q2. Input/Output

Objective: Using cin, cout

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Enter your age: ";
    cin >> age;
    cout << "Your age is: " << age << endl;
    return 0;
}
```

OUTPUT:

```
Enter your age: 20
Your age is: 20
```

Q3. Swap two numbers

Objective: Using temp / without temp

```
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 10, temp;
    cout << "Before swap: a = " << a << ", b = " << b << endl;
    temp = a;
    a = b;
    b = temp;
    cout << "After swap (with temp): a = " << a << ", b = " << b << endl;
    return 0;
}
```

OUTPUT:

```
Before swap: a = 5, b = 10
After swap (with temp): a = 10, b = 5
```

Q4. Prime Number

Objective: Check for prime

```
#include <iostream>
using namespace std;

int main() {
    int n = 29;
    bool isPrime = true;
    for(int i = 2; i <= n/2; ++i) {
        if(n % i == 0) {
            isPrime = false;
            break;
        }
    }
    if (isPrime && n > 1) cout << n << " is a prime number." << endl;
    else cout << n << " is not a prime number." << endl;
    return 0;
}
```

OUTPUT:

```
29 is a prime number.
```

Q5. Factorial

Objective: Using loop or recursion

```
#include <iostream>
using namespace std;

int main() {
    int n = 5, fact = 1;
    for(int i = 1; i <= n; ++i) {
        fact *= i;
    }
    cout << "Factorial of " << n << " is " << fact << endl;
    return 0;
}
```

OUTPUT:

```
Factorial of 5 is 120
```

Q6. Even or Odd

Objective: Check if a number is even or odd

```
#include <iostream>
using namespace std;

int main() {
    int n = 15;
    if(n % 2 == 0) cout << n << " is Even." << endl;
    else cout << n << " is Odd." << endl;
    return 0;
}
```

OUTPUT:

```
15 is Odd.
```

Q7. Leap Year Checker

Objective: Check if year is leap

```
#include <iostream>
using namespace std;

int main() {
    int year = 2024;
    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        cout << year << " is a leap year." << endl;
    else
        cout << year << " is not a leap year." << endl;
    return 0;
}
```

OUTPUT:

```
2024 is a leap year.
```

Q8. Calculator

Objective: Basic arithmetic operations

```
#include <iostream>
using namespace std;

int main() {
    char op = '+';
    double num1 = 12, num2 = 5;
    cout << "Operation: 12 + 5 = ";
    if(op == '+') cout << num1 + num2 << endl;
    return 0;
}
```

OUTPUT:

```
Operation: 12 + 5 = 17
```

Q9. Reverse a number

Objective: Reverse using loop

```
#include <iostream>
using namespace std;

int main() {
    int n = 1234, reversedNum = 0, remainder;
    while(n != 0) {
        remainder = n % 10;
        reversedNum = reversedNum * 10 + remainder;
        n /= 10;
    }
    cout << "Reversed Number = " << reversedNum << endl;
    return 0;
}
```

```
OUTPUT:
Reversed Number = 4321
```

Q10. Palindrome

Objective: Check numeric/string palindrome

```
#include <iostream>
using namespace std;

int main() {
    int n = 121, reversedNum = 0, remainder, originalNum;
    originalNum = n;
    while(n != 0) {
        remainder = n % 10;
        reversedNum = reversedNum * 10 + remainder;
        n /= 10;
    }
    if (originalNum == reversedNum) cout << originalNum << " is a palindrome." << endl;
    else cout << originalNum << " is not a palindrome." << endl;
    return 0;
}
```

```
OUTPUT:
121 is a palindrome.
```

Q11. Fibonacci

Objective: Recursive and loop

```
#include <iostream>
using namespace std;

int main() {
    int n = 5, t1 = 0, t2 = 1, nextTerm = 0;
    cout << "Fibonacci Series: ";
    for (int i = 1; i <= n; ++i) {
        cout << t1 << " ";
        nextTerm = t1 + t2;
        t1 = t2;
        t2 = nextTerm;
    }
    cout << endl;
    return 0;
}
```

OUTPUT:

```
Fibonacci Series: 0 1 1 2 3
```

Q12. GCD/LCM

Objective: Using Euclidean algorithm

```
#include <iostream>
using namespace std;

int main() {
    int a = 12, b = 18;
    int x = a, y = b;
    while(x != y) {
        if(x > y) x -= y;
        else y -= x;
    }
    cout << "GCD of " << a << " and " << b << " is " << x << endl;
    cout << "LCM of " << a << " and " << b << " is " << (a * b) / x << endl;
    return 0;
}
```

OUTPUT:

```
GCD of 12 and 18 is 6
LCM of 12 and 18 is 36
```

Q13. Class and Object

Objective: Create and use simple class

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    void display() {
        cout << "Student Name: " << name << endl;
    }
};

int main() {
    Student s1;
    s1.name = "Rahul";
    s1.display();
    return 0;
}
```

OUTPUT:

```
Student Name: Rahul
```

Q14. Constructor & Destructor

Objective: Use default, parameterized, and copy constructors

```
#include <iostream>
using namespace std;

class Example {
public:
    int x;
    Example() { x = 0; cout << "Default Constructor called" << endl; }
    Example(int a) { x = a; cout << "Parameterized Constructor called" << endl; }
    ~Example() { cout << "Destructor called for x = " << x << endl; }
};

int main() {
    Example e1;
    Example e2(50);
    return 0;
}
```

OUTPUT:

```
Default Constructor called
Parameterized Constructor called
Destructor called for x = 50
Destructor called for x = 0
```

Q15. Access Modifiers

Objective: Use of private, public, protected

```
#include <iostream>
using namespace std;

class Base {
private:
    int priv = 10;
protected:
    int prot = 20;
public:
    int pub = 30;
    void show() {
        cout << "Private: " << priv << ", Protected: " << prot << ", Public: " << pub <<
    }
};

int main() {
    Base b;
    b.show();
    return 0;
}
```

OUTPUT:

```
Private: 10, Protected: 20, Public: 30
```

Q16. Encapsulation Example

Objective: Bundle data and functions

```
#include <iostream>
using namespace std;

class Employee {
private:
    int salary;
public:
    void setSalary(int s) { salary = s; }
    int getSalary() { return salary; }
};

int main() {
    Employee e;
    e.setSalary(50000);
    cout << "Salary: " << e.getSalary() << endl;
    return 0;
}
```

OUTPUT:

```
Salary: 50000
```

Q17. Data Abstraction

Objective: Expose essential features only

```
#include <iostream>
using namespace std;

class AbstractClass {
public:
    virtual void display() = 0;
};

class ConcreteClass : public AbstractClass {
public:
    void display() override {
        cout << "Showing implementation detail hides from user." << endl;
    }
};

int main() {
    ConcreteClass obj;
    obj.display();
    return 0;
}
```

OUTPUT:

```
Showing implementation detail hides from user.
```

Q18. Inline Functions in Class

Objective: Declare and define inline functions inside a class

```
#include <iostream>
using namespace std;

class InlineDemo {
public:
    inline void greet() {
        cout << "Hello from inline function inside class!" << endl;
    }
};

int main() {
    InlineDemo obj;
    obj.greet();
    return 0;
}
```

```
OUTPUT:
Hello from inline function inside class!
```

Q19. Static Data Members & Functions

Objective: Use shared variables across objects

```
#include <iostream>
using namespace std;

class Counter {
public:
    static int count;
    Counter() { count++; }
    static int getCount() { return count; }
};

int Counter::count = 0;

int main() {
    Counter c1, c2;
    cout << "Total objects created: " << Counter::getCount() << endl;
    return 0;
}
```

```
OUTPUT:
Total objects created: 2
```

Q20. Friend Function

Objective: Access private members using friend

```
#include <iostream>
using namespace std;

class Box {
private:
    int width;
public:
    Box() : width(15) {}
    friend void printWidth(Box b);
};

void printWidth(Box b) {
    cout << "Width of box: " << b.width << endl;
}

int main() {
    Box b;
    printWidth(b);
    return 0;
}
```

OUTPUT:

```
Width of box: 15
```

Q21. This Pointer

Objective: Use this to resolve name conflicts

```
#include <iostream>
using namespace std;

class Test {
    int x;
public:
    void setX(int x) {
        this->x = x;
    }
    void print() { cout << "x = " << x << endl; }
};

int main() {
    Test t;
    t.setX(25);
    t.print();
    return 0;
}
```

OUTPUT:

```
x = 25
```

Q22. Const Member Functions

Objective: Declare member functions that don't modify data

```
#include <iostream>
using namespace std;

class Demo {
    int value;
public:
    Demo(int v) : value(v) {}
    void show() const {
        cout << "Value: " << value << endl;
    }
};

int main() {
    Demo d(100);
    d.show();
    return 0;
}
```

```
OUTPUT:
Value: 100
```

Q23. Single Inheritance

Objective: One class inherits another

```
#include <iostream>
using namespace std;

class Parent {
public:
    void parentMethod() { cout << "Inherited from parent." << endl; }
};

class Child : public Parent {};

int main() {
    Child c;
    c.parentMethod();
    return 0;
}
```

```
OUTPUT:
Inherited from parent.
```

Q24. Multiple Inheritance

Objective: A class inherits from two classes

```
#include <iostream>
using namespace std;

class A { public: void showA() { cout << "Class A" << endl; } };
class B { public: void showB() { cout << "Class B" << endl; } };
class C : public A, public B {};

int main() {
    C obj;
    obj.showA();
    obj.showB();
    return 0;
}
```

OUTPUT:

```
Class A
Class B
```

Q25. Function Overloading

Objective: Same function name with different arguments

```
#include <iostream>
using namespace std;

class Display {
public:
    void print(int i) { cout << "Printing int: " << i << endl; }
    void print(double f) { cout << "Printing float: " << f << endl; }
};

int main() {
    Display d;
    d.print(5);
    d.print(5.5);
    return 0;
}
```

OUTPUT:

```
Printing int: 5
Printing float: 5.5
```